The Artifact Promotion Control Model: An Implementation Case Study

Build on Target Machines vs. Build Once and Promote Artifacts

Vasili Gavrilov Enkli Ylli, PhD, CISSP, CCSP

First version 3 June 2026; revised 25 August and 2 September 2026

Abstract

A previous treatment by the first author [1] presented artifact promotion as a control model for cloud deployment, comparing build-on-target with build-once-and-promote in accessible engineering prose. This paper has two parts. Part I restates the control model in stricter form, covering the release object, the control domains a deployment crosses, the artifact-versus-environment identity distinction, source-control compromise as a control-domain question, and the regulatory frameworks under which the model's properties become structurally required rather than merely preferable. Part II is an anonymized implementation case study of a production web application on Amazon Web Services, in which the only manual deployment step is the upload of a versioned artifact to an S3 bucket and every subsequent stage runs autonomously. We report end-to-end deployment timings from a development-environment trial.

Contributions. Beyond restating the model, the paper (1) characterizes promotion as layer-independent, holding equally for container images, virtual-machine images, operating-system packages, and versioned archives; (2) separates secrets *mechanism* from secrets *timing* and shows that build-time secret injection is incompatible with promotion; (3) frames deployment-time dependency resolution as an adversarial supply-chain surface that scales with the dependency closure; (4) derives a separation of release authority from runtime-secret authority; (5) translates the cited compliance frameworks (FedRAMP/SI-7, SOX 404, FFIEC, DO-178C, FDA 21 CFR Part 11, HIPAA, DoD IL) into the concrete obligations they place on engineers, not only auditors; and (6) reports end-to-end deployment timings from a trial. The full list of contributions appears at the end of the Preface.

Preface: Why Now

The reported volume of software vulnerabilities has set a record for seven consecutive years: the CVE program published just over 40,000 vulnerabilities in 2024, roughly 38% more than in 2023, and submissions have grown fast enough to backlog the National Vulnerability Database. [10] Against this background, software supply-chain attacks, which target the build and distribution path rather than a flaw in the running application, are a documented and growing class: the European Union Agency for Cybersecurity has reported a marked rise in their number and sophistication [9], and a peer-reviewed systematization catalogues 107 distinct attack vectors drawn from 94 real-world incidents [7]. The deployment path is an increasing share of the attack surface, not a static one.

These attacks repeatedly exploit access to source code or to third-party build inputs at deployment time. In March 2025, a widely consumed continuous-integration action was compromised: its version

tags were retroactively repointed to malicious code that extracted secrets from the runner process and wrote them, encoded, into the workflow logs, where on public repositories anyone could read them. [6] Over twenty thousand repositories depended on the action. The point is not that one specific package was breached; it is that under any deployment model where the production execution path consumes source code or third-party build inputs at deployment time, a single upstream compromise becomes a production incident on the next rollout.

The implications for systems where deployment failures translate into safety failures, such as aviation control, infusion pumps, medical imaging, and industrial process control, are not hypothetical. They are why the regulatory regimes governing those systems require deployment evidence that build-on-target-machine models produce only with substantial additional effort, if at all. Part I of this paper makes the architectural argument and enumerates the relevant frameworks. Part II documents what the implementation actually looks like, how long it takes, and what is operationally easy to get wrong.

Contributions beyond restatement. (i) A layer-independent characterization of artifact promotion that treats container-image registries, virtual-machine image repositories, operating-system package repositories, and versioned-archive object stores as equally valid implementations of the same model. (ii) A distinction between secrets *mechanism* (protected file vs. managed secrets store, comparable security posture) and secrets *timing* (runtime fetch vs. build-time injection); the argument that build-time injection is incompatible with the model because it embeds environment-specific state into the artifact and breaks the property that the same artifact is promotable across environments. (iii) An argument that deployment-time dependency resolution against public package registries is both a non-adversarial failure mode, through upstream removal and resolution non-determinism, and an adversarial code-injection surface whose probability of compromise scales with the size of the dependency closure; and that promotion moves dependency resolution out of the production path and reduces it from a per-deploy, per-instance exposure to a single reviewable build-time event. (iv) Empirical measurements from a trial deployment: end-to-end deployment timing on a single-instance environment, broken down by pipeline stage. (v) A short list of widespread naming hazards in managed-platform vocabulary that mislead engineers reading this kind of architecture for the first time. (vi) A statement of three operational properties that follow from the artifact-identity boundary: rollback as the selection of a known artifact rather than a rebuild, the structural prevention of mixed-version fleets arising from partial deployment, and a reduced production attack surface. (vii) A translation of the cited compliance frameworks into the concrete deployment obligation each places on the engineering team rather than on the auditor, retaining the control identifiers a compliance reader recognizes, together with a non-regulatory case in which the same byte-identity property is wanted for performance rather than for compliance. (viii) A separation-of-duties property that follows from the artifact carrying no secrets: release authority, the production and promotion of an approved artifact, and runtime-secret authority, the provisioning of each environment's secrets, become distinct roles that need not be held by the same person, so a release can be triggered without secret access and a second party can roll back or redeploy.

Part I: The Control Model

1 The Release Object

A common framing of the release problem is the question:

> Did we build the latest code on production?

A stricter framing is:

> Which approved artifact is currently deployed?

The shift is not rhetorical. "Latest code" is a property of a version-control repository and changes whenever a commit lands. An *approved artifact* is a concrete object: it has a version label, an integrity hash, a source revision identifier, a build timestamp, an approval record, a deployment history, and a rollback path. Engineers, project managers, testers, auditors, and stakeholders can reason about a single object with these attributes. They cannot reason coherently about "the latest code" because each of them may be looking at a different moving snapshot of it.

In the model of this paper, the release is the artifact selected for deployment. The release is not an informal statement that production built something from the source repository.

2 Two Deployment Models

Environment naming conventions vary across organizations: development, integration, quality assurance, staging, user acceptance, pre-production, production, disaster recovery, and customer-specific replicas all appear in production usage, with per-organization semantics. A deployment model that depends on those names is not transferable. The model considered here depends only on a more fundamental property: where in the system topology does the build occur?

Model A: build on target. Each deployment instance retrieves the source revision corresponding to the intended release, resolves the application's dependency closure against external sources, performs the build on the instance, and deploys the locally produced bytes.

Model B: build once and promote. A single controlled build, performed in a designated environment, produces a deployable artifact identified by a version label and a content hash. The same artifact is then stored, identified, approved, and promoted through deployment environments without rebuild.

```
Model A:
    source repository -> target instance -> local build -> local deploy -> runtime

Model B:
    source repository -> controlled build -> artifact store -> deployment -> runtime
    configuration -> runtime
```

Model A is fine for local development, prototypes, single-machine systems, and short-lived test hosts. Its limitation is that it collapses several responsibilities (source checkout, dependency resolution, build, deployment placement, runtime execution) onto each target instance, which makes the separations required for controlled release operationally difficult.

Model B makes the release boundary explicit. The production instance receives a known deployable unit. It does not create the release by compiling source during deployment.

A consequence concerns how each model selects an environment. Under Model B the environment is determined by which artifact and which runtime configuration are deployed, so a single source trunk can remain the one source of truth. Under Model A, teams frequently grow per-environment branches to carry the differences, and those branches drift: a fix cherry-picked into one is missed in another, and the question *what is in production?* becomes a forensic exercise rather than a lookup.

```
Model A tendency                        Model B
  main                                    main   (single source of truth)
   |  cherry-pick (drifts)                 |
   +--> dev  branch                        |   one controlled build
   +--> qa   branch  <- picks missed       v
   +--> prod branch  <- picks missed       artifact-X.Y.zip
                                             |  promoted unchanged
  "what is in prod?" -> forensic            +--> DEV -> UAT -> PROD
                                            "what is in prod?" -> an artifact id
```

Architects and engineers frequently treat these two models as interchangeable deployment plumbing, which is why the distinction needs drawing. The difference is more than architectural preference: building binaries once and deploying the same artifact to every environment is a continuous-delivery practice that the multi-year DevOps Research and Assessment program associates, in its published research, with higher software-delivery performance [3]. The control properties developed through the rest of Part I depend on the build sitting off the target; they are lost, quietly, the moment it moves on.

3 Control Domains and the Promotion Boundary

Artifact promotion rests on a partition of responsibilities that Model A collapses:

```
Source repository      :  code revision history
Build subsystem        :  artifact creation
Artifact store         :  release identity
Deployment subsystem   :  controlled promotion
Runtime environment    :  configuration and secrets
Application instance   :  execution
```

These are control domains in the security-architecture sense: each has a distinct ownership, a distinct audit trail, and a distinct compromise scope. The central architectural claim is that source-code evolution and production change should be operated in different control domains, and that the promotion boundary is the interface between them.

The build subsystem may run on any controlled environment, from a release master's workstation to a hosted CI service; the model is defined not by the product but by building the deployable unit once, giving it a stable identity, and promoting it.

The published account of the model in the binary-repository-manager literature (notably the JFrog Artifactory and Sonatype Nexus documentation) names this verb *promotion* and treats the artifact store as the system of record for the release identity. *Continuous Delivery* [2] formalized the surrounding pipeline; the *Twelve-Factor App* [4] canonicalized the three-stage build / release / run separation. The integrity properties the promotion boundary depends on, content-addressable

identity, cryptographic signing, and verifiable provenance, are the subject of an active supply-chain-security literature, including the SLSA framework [11], in-toto attestations [12], the Sigstore signing infrastructure [13], and the reproducible-builds effort [14]. The case study in Part II is consistent with the vocabulary of those sources.

4 Artifact Formats and the Layer of Promotion

The artifact format is an implementation choice within the model. The deployable unit may be any of the following, among others:

- A versioned application archive (zip, tarball, or vendor package) deployed onto a prepared instance with a pre-installed runtime; the format used in Part II.
- A container image (OCI / Docker) bundling code, dependencies, and a minimal user-space runtime, promoted through a registry and run by a container engine, often under Kubernetes.
- A virtual-machine image (e.g. an AMI) bundling code, libraries, runtime, and an OS kernel, promoted through an image repository.
- An operating-system package (deb, rpm, msi), promoted through a package repository.
- A language-runtime archive (jar, war, wheel, gem, dll, exe), promoted through a language-specific repository.
- A filesystem or block-device snapshot of a prepared instance, promoted through the platform's snapshot service.

Each format draws a different boundary between what is in the artifact and what is assumed present on the host, but the control-domain claims apply to all of them: each has a stable identity (a content hash and a version label), each can be hashed, signed, scanned, approved, and archived independently of source control, and each promotes across environments without rebuild. The choice among them turns on artifact size, host-preparation cost, isolation needs, and the regulatory frameworks discussed below, not on the model.

No single format is privileged by the model: the artifact format is simply the layer at which promotion is applied. A reader whose deployment process already promotes any of the formats listed above, whether container images through a registry or versioned archives through an object store, is already using the model.

5 Environment-Scoped Runtime Configuration

A second separation, distinct from the source/build/artifact partition, is between artifact identity and environment identity.

The source repository contains application code that is identical for every deployment environment. The artifact contains built application bytes and deployment scripts that are identical for every deployment environment. Environment-specific runtime data is in neither. It has two categories:

- **Secrets:** production database passwords, third-party API credentials, encryption keys, OAuth client secrets, mail-transport credentials, and similar values whose disclosure to the wrong parties causes immediate operational harm.

- **Operational configuration:** database connection strings, internal service URLs, content-delivery distribution identifiers, object-store bucket names, feature flags, regional identifiers, environment markers. These values may not be secret in the strict sense, but they define the environment in which the artifact runs.

Both categories belong to the runtime environment, not to source control and not to the artifact. The implementation rule is that *the artifact does not know in which environment it is running*; it starts on a prepared instance and reads the appropriate configuration from a single authoritative source.

The mechanism may be simple or sophisticated: a protected file on the instance's local disk; a configuration-management system; a managed secrets store; an attached configuration volume in a container platform. The architectural property is the same regardless of mechanism: same source, same artifact, different runtime configuration per environment.

A further distinction is between secrets resolved at runtime on the instance and secrets injected into the artifact at build time. Runtime resolution leaves each secret in one place, the chosen store; build-time injection copies it into the artifact as well, and two copies leak more easily than one. Injection also breaks artifact identity: an artifact carrying one environment's secrets is no longer the same artifact deployed elsewhere, so the "same artifact, different runtime configuration per environment" property is lost. Whether a secret lives in a protected file or a managed store is an implementation choice of comparable security posture; whether it is resolved at runtime or injected at build time is a model-level decision that determines whether the artifact stays promotable. Part II resolves all secrets at runtime for this reason.

Model A does not logically require secrets in source control, and a disciplined team can run it with runtime configuration correctly; the objection is structural, not ideological. When every target instance performs source checkout, dependency resolution, build-profile selection, compilation, and deployment, the boundaries among source, build configuration, deployment configuration, and runtime configuration are easier to blur. Environment-specific files, branches, profiles, scripts, and ad-hoc overrides accumulate near the source checkout, even when no operator intends it, and raise the risk of secret sprawl and accidental commits.

Separation of release authority from runtime-secret authority. Because the artifact contains no environment-specific secrets, the person who builds and promotes it needs no access to any environment's secrets in order to release. This yields a separation of duties that the model provides almost for free. The release master produces an approved artifact and triggers its promotion but need hold no production credentials and need not read any runtime secret; those are provisioned separately by an environment administrator into the per-environment configuration source. Release authority and runtime-secret authority become distinct roles, held by different people. The consequence is operational as well as security-related: because a release is the selection of an identified artifact rather than a privileged build by one individual, a second person with environment access can roll back or redeploy without involving the release master. Under Model A, where deploying is itself a build run with the deploying identity's access, this separation is much harder: deploying tends to demand the same broad access as operating the environment.

6 Source-Control Compromise and Control-Domain Separation

If source control is compromised, every deployment model requires incident response. Promotion does not make malicious code impossible. An attacker who can alter source can affect a future build.

The difference between the two models is the control path from a source compromise to a production incident. Under Model A, production instances consume the repository directly at deployment time. If deployment proceeds by source pull followed by local build, source control is part of the production execution path on every deploy. A compromise of the source repository becomes a production compromise on the next deployment cycle.

Under Model B, production receives a previously produced artifact whose identity can be hashed, signed, scanned, approved, and archived. A source-control compromise still affects a future build, but it does not by itself rewrite the identity of an artifact already in production, reveal runtime secrets, or force production to compile from the current repository state. Source code, artifact identity, deployment approval, and runtime secrets sit in separate control domains. Compromise of one does not propagate immediately to the others.

This is the security property of the model: not that source control can never be compromised, but that source-control compromise has a more contained and more auditable blast radius, rather than an immediate path to runtime execution.

7 Operational Properties: Rollback, Fleet Consistency, and Production Surface

Three operational properties follow directly from the artifact-identity boundary, and they are worth stating explicitly because they are the properties an operations team feels first.

Rollback is artifact selection, not rebuild. Under Model B, reverting a bad release is the redeployment of a previously approved artifact that is still present, by identity, in the artifact store. The bytes that ran yesterday are the bytes that run again; no compilation is involved and the rollback target is known exactly. Under Model A, rollback means selecting an earlier source revision and rebuilding it on every target instance, with the same toolchain-drift exposure as the original deployment and no guarantee that today's rebuild of last week's revision reproduces last week's bytes. Rollback under Model A is therefore itself a fresh build event rather than a return to a known state.

Partial-fleet deployment cannot silently split the fleet. Consider a ten-instance fleet. Under Model A, each instance independently checks out source, resolves dependencies, and builds. If seven succeed and three fail, on a transient network fault, a dependency-registry outage, or a local disk or package-state problem, the fleet is left running two different versions of the application behind one load balancer, and the split is not visible from any single build log. Under Model B, every instance receives the same already-built artifact; an instance either has that artifact and runs it or does not start. The mixed-version fleet is structurally prevented rather than monitored for.

The production host carries less. Model A requires a source-control client, a language runtime sufficient to *build*, not only to run, a build tool, and a working source checkout to be present on every production instance. Each is an independent patching obligation, an addition to the host's vulnerability surface, and a potential source of information disclosure if an instance is compromised.

Model B requires only the application runtime and a deployment procedure; the build toolchain and the source checkout are absent from production entirely. A smaller production surface is easier to harden, to patch, and to reason about. A related availability property follows: because Model A places a source-control client and a dependency resolver on the deploy path, a source-control outage, an expired credential, a branch-protection change, or a blocked network egress can prevent a deployment; under Model B the approved artifact is already present and installs without reaching any external system.

8 Regulatory Frameworks Under Which the Model's Properties Become Required

The control-domain properties documented above become structurally required, rather than merely preferable, in several regulatory contexts. The frameworks below are written in the vocabulary of auditors, but the obligations they create land on the engineers who build and operate the deployment. Each entry therefore states three things: what the regime governs, the control most relevant to the deployment question, and, in concrete terms, what the team running the pipeline must be able to demonstrate at deploy time and why build-on-target deployment cannot produce it.

FedRAMP and NIST SP 800-53 control SI-7. FedRAMP authorizes cloud services that handle U.S. federal data; its baselines incorporate the controls of NIST Special Publication 800-53. [5] The control SI-7 ("Software, Firmware, and Information Integrity") requires the system operator to detect unauthorized changes to executable code and configuration, in practice by hash-verifying every artifact at deploy time against a known-good value recorded at build time. Under Model A there is no single hashable artifact, because the artifact is reconstructed per machine from a moving source. For the engineer operating the pipeline, SI-7 reduces to a question that must have an answer at any moment: what are the exact bytes on this instance, and do they match an approved build? Promotion answers it by recording the artifact's hash once at build time, storing it with the release identity, and re-verifying it before the instance accepts the artifact; under Model A the question has no single stable answer, because each host builds its own bytes from a local toolchain and leaves nothing fixed to verify against.

Sarbanes–Oxley Section 404. The U.S. Sarbanes–Oxley Act of 2002 requires public-company management to assess the effectiveness of internal controls over financial reporting. Section 404 in particular drives segregation-of-duties controls between code authors and production operators, a standard mitigation against unauthorized changes to financial systems. Model A tends to collapse the boundary: granting a production machine the authority to compile arbitrary remote source effectively grants every code author production-execution rights, which complicates the segregation-of-duties posture. In operational terms, this is why the person who writes the code must not also be the one who silently lands it on the systems of record. Promotion makes the separation mechanical: a commit, the approved artifact, and the promotion step each carry their own owner and their own audit trail, so the control the auditor wants to see is the control the pipeline already enforces.

FFIEC IT Examination Handbook. The Federal Financial Institutions Examination Council sets examination standards for U.S. banking regulators. Its IT Examination Handbook expects controlled, auditable change management with a clear development, test, and production separation. The artifact-promotion model produces the change-management evidence that examinations look for: artifact identity, approval record, deployment time, rollback record. Model A produces equivalent evidence only with substantial additional process overhead. For the team, the difference is

concrete: when an examiner asks to see the change record for what is currently in production, promotion answers with an artifact identity, an approval, a deploy timestamp, and a rollback record, rather than a history reconstructed from per-machine build logs after the fact.

RTCA DO-178C. DO-178C is the software-certification standard used by civil aviation authorities for software in airborne systems. It mandates traceability between requirements, source code, executable object code, and the verification record. Certification accepts a specific, identified, and frozen executable; recompiling on each target during deployment invalidates the certification basis. Model A is very difficult to reconcile with DO-178C. For the build-and-release engineer the reason is direct: the certified entity is one frozen executable with a verification record bound to those exact bytes, so a deployment that recompiles on each target ships bytes that were never the verified ones.

FDA 21 CFR Part 11. The U.S. FDA's regulation on electronic records and signatures governs software used in pharmaceutical, biotech, and medical-device manufacturing and quality processes. It requires records of system changes that are "trustworthy, reliable, and ... equivalent to paper records," which presumes an identifiable, reviewable change unit. The deployment unit must be the artifact, with its identity, its approval record, and its associated validation runs. A per-machine recompile produces records that are not equivalent across the fleet. For the operator this means the deployable unit must be the same unit that was reviewed and validated, namely the artifact together with the validation runs executed against those exact bytes, not a fresh per-host compile whose bytes no electronic record describes.

HIPAA Security Rule. The HIPAA Security Rule requires covered entities to implement administrative, physical, and technical safeguards for electronic protected health information. The technical-safeguard category includes integrity controls, which in the software deployment context map to the same artifact-identity requirement as in SI-7. For the engineer responsible for a system handling protected health information, the practical obligation is identical to SI-7: be able to show that the bytes serving patient data are the approved bytes, not whatever a target host happened to compile.

Department of Defense Impact Levels IL-4 through IL-6. The Department of Defense classifies cloud information by sensitivity (Controlled Unclassified Information at IL-4 and IL-5; classified information at IL-6). Higher levels operate in network enclaves with restricted or no public-network egress. Cross-domain transfer of deployable code at these levels accepts artifacts, not source repositories; Model A is operationally infeasible at IL-5 and IL-6 in the common case. The reason is concrete: the production network typically cannot reach a public source-control system or dependency registry, so a model that checks out source and resolves dependencies at deploy time has nothing to reach, and in practice only a self-contained approved artifact crosses the boundary.

The artifact-promotion model does not by itself produce compliance with any of these frameworks; compliance is a much larger system property. The point of the enumeration is that the control boundaries required by these frameworks are the same boundaries the model establishes, and that operating teams who expect to be evaluated under any of them benefit from establishing the boundaries early rather than retrofitting them under audit pressure.

Beyond regulated domains. Build determinism is also valuable where it carries no compliance mandate but a direct operational cost. In latency-sensitive systems such as high-frequency trading, where binaries are tuned with specific optimization flags, profile-guided or ahead-of-time compilation, and pinned dependency versions, a per-target rebuild on heterogeneous build hosts can introduce measurable timing variance between instances that are nominally running the same release. Promoting one tuned artifact removes that variance by construction. Here the defining property of the model, one identified set of bytes everywhere, is wanted for performance reasons rather than for audit

reasons.

9 Common Objections

"Old artifacts ship vulnerabilities." Old artifacts should not be promoted indefinitely. Vulnerability response is cleaner under promotion: rebuild once, identify the new artifact, scan it, approve it, and promote it. The alternative is coordinating source pulls and local builds across a fleet, with no way to verify that all instances built equivalent bytes.

"This is overengineering." The mechanism can be simple. An artifact store, a versioned package, a deployment procedure, and an environment-scoped runtime configuration set are sufficient to establish the control boundary. The complexity of larger implementations is a property of the systems they support, not of the model.

"Source on the server helps debugging." Production debugging should use logs, metrics, traces, symbols, source maps, source archives, or controlled diagnostic packages. Keeping a source checkout on production is not the same as having a stable release process, and the two should not be conflated.

"Promotion means committing to one CI vendor." It does not. The controlled build can run on a release master's workstation, Jenkins, GitHub Actions, GitLab CI, Azure Pipelines, AWS CodeBuild, or any other controlled environment. The model is defined by building once, giving the artifact a stable identity, and promoting it; it is not defined by the product that performs the build. The case study in Part II happens to build on a workstation and to deploy through AWS CodeBuild used purely as a script runner, but nothing in the model requires either choice.

Part II: Implementation Case Study

10 Setting

The system under study is a production web application: a backend exposing a REST API, a static frontend served from a content-delivery network, and a relational database in a managed relational-database service. The backend's implementation language is immaterial to everything that follows; the recipe concerns the deployment mechanics, not the application. The fleet consists of stateless application instances behind an application load balancer. Version control follows a single shared trunk with annotated release tags. The team uses pull-often, push-only-what-compiles discipline rather than long-lived feature branches.

The infrastructure is hosted on Amazon Web Services. The case study names the specific services (Amazon S3, EventBridge, CodeBuild, Elastic Beanstalk, behind an Application Load Balancer and CloudFront) because a recipe is only useful if concrete; equivalents exist on other platforms, and Part I's argument is platform-independent. Withheld is anything that would grant privileged access or identify the deployment: account identifiers, IAM principal and role names, bucket names, CloudFront distribution identifiers, and hostnames. Topology, control flow, and the exact API actions are preserved; no nameable resource or credential is.

Three deployment environments are in scope: a development environment used for staged trials

of the new model, and two reference environments (user-acceptance testing and production) that are still operated under the predecessor pipeline and will migrate later. This paper reports on the development-environment implementation in detail.

11 Materials and Methods

11.1 The deployment pipeline

The deployment pipeline is invoked by a single command on the release master's machine: a release build that produces a versioned zip artifact, followed by an `aws s3 cp` upload of that artifact to an environment-specific S3 bucket. The upload is the only laptop-side action. Everything else runs autonomously in the cloud.

```
Release master (local):
    release build  produces  artifact-<VER>.zip   (built locally; no cloud build)
    aws s3 cp      artifact-<VER>.zip -> s3://<artifact-bucket-for-env>
        |
        | S3 ObjectCreated event
        v
Amazon EventBridge rule (one per environment)
        | fires StartBuild on the CodeBuild project
        v
AWS CodeBuild project (source = NO_SOURCE)
        | downloads the artifact from S3
        | runs the deployment script (buildspec):
        |     - sync static assets to the CloudFront origin bucket
        |     - create a CloudFront invalidation
        |     - assemble the Elastic Beanstalk source bundle from the artifact
        |     - upload bundle, CreateApplicationVersion, UpdateEnvironment
        v
AWS Elastic Beanstalk
        | rolls each EC2 instance to the new application version
        v
Per EC2 instance, at predeploy (.platform hook):
        | the hook probes each candidate config bucket in S3;
        |   the instance role's IAM policy grants s3:GetObject on
        |   exactly one bucket - the one for this instance's environment
        | pulls the config files to local paths, sets owner and mode
        v
Application restarts and reads its config on startup
```

The release build also records each artifact's identity: it computes a checksum and stamps a build time, so every promoted zip carries the content hash and build timestamp named in the Release Object section. Publishing a checksum beside a distribution is long-standing practice for serious software releases, and here it is the concrete realization of the integrity-hash requirement behind controls such as SI-7: the value recorded at build time is the known-good value an instance can verify against at deploy time, and the same value identifies the artifact through promotion and rollback. For tamper-evidence specifically, the recorded hash should be collision-resistant (SHA-256 or stronger); a faster non-cryptographic checksum still serves to catch accidental corruption in transit.

11.2 CodeBuild triggers and runs the deployment; it does not build the application

The CodeBuild project is configured with its source set to `NO_SOURCE`. This is an easily missed but important point about the pipeline, and the AWS product name works against it: *no compilation occurs in this stage.* The application artifact has already been built on the release master's machine before the upload. CodeBuild is used here purely as an event-triggered, IAM-scoped script runner: it downloads the existing artifact and executes the deployment script (its `buildspec`) against the AWS control plane. It compiles nothing, checks out no source, and resolves no dependencies. Engineers reading the architecture for the first time consistently assume the build happens here and look in the wrong place when diagnosing problems; any documentation derived from this pattern should state on first mention that CodeBuild is the trigger-and-deploy mechanism, not a build step.

11.3 Dependency materialization

The release artifact contains every third-party library the application needs to run. Nothing is resolved at deployment time, and nothing is resolved at instance start, against any internet-hosted dependency repository. Operators new to the pattern often question this choice in favor of letting the build host resolve transitive dependencies on demand. The case against is operational rather than ideological.

Build tooling that pulls transitive dependencies from public registries is vulnerable to two failure modes that have nothing to do with the supply-chain security incidents discussed in the Preface. First, the maintainer of a transitive dependency may withdraw a published version. A previously successful dependency tree then ceases to resolve. The author observed exactly this when a published REST client library's declared closure became unbuildable after one upstream transitive removal, blocking work that had compiled cleanly days before. Second, transitive resolution introduces nondeterminism in builds across time: two builds of the same source revision performed weeks apart may produce different byte sequences because the resolver selected different versions of a transitive in the interim. Neither failure mode requires an adversary; both arise from ordinary maintenance activity on the open registry.

A third consideration is adversarial and connects directly to the Preface. Every public package registry from which dependencies are resolved (Maven Central, npm, PyPI, and their equivalents) is a code-injection surface: a compromised or maliciously republished package executes in whatever environment resolves it. This risk scales with the size of the dependency closure. A project that declares a handful of direct dependencies transitively pulls tens or hundreds of packages, each maintained by a different party with a different security posture, and the probability that at least one is compromised at any given moment rises with their number. Malicious packages are a documented and recurring class of supply-chain attack, catalogued in peer-reviewed reviews of real-world incidents across npm, PyPI, and other registries [8, 7]. Under Model A, that resolution happens at deploy time, on or near production, on every rollout, so each deployment re-exposes the production path to the current state of every upstream package. Under Model B, dependency resolution happens once, at controlled build time and away from production; the resolved set is captured in the artifact, can be scanned and hash-pinned before approval, and is not re-fetched from any registry at deploy time. Promotion does not eliminate dependency-supply-chain risk, but it moves the resolution out of the production path and reduces it from a per-deploy, per-instance exposure to a single reviewable build-time event.

The implementation here treats every library the application needs as part of the artifact, in the same sense that a statically linked binary in a systems-programming language contains the libraries it links against rather than deferring resolution to a shared-library directory that may have changed. Library updates are deliberate edits to the artifact's bundled set, reviewed at the same level of attention as application code changes. The trade-off is a larger artifact and the loss of "free" upstream patches; the gain is byte-deterministic builds from a given source revision and robustness against upstream removals.

11.4 Environment identity via IAM scoping

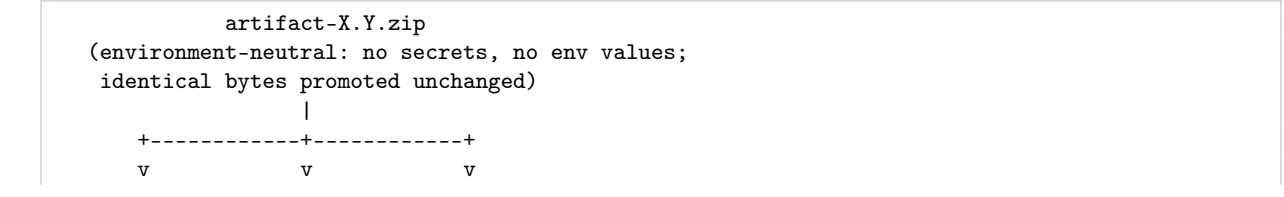
Each instance must identify the environment in which it is running so that it can fetch the correct configuration. Two common alternatives are rejected here on operational grounds, not security ones. An operating-system environment variable adds a second source of truth that can drift from the bucket-name reality, plus a templating step on each deploy. A managed secrets store adds latency and a credential lifecycle just to learn one string; its security is comparable to a properly protected file, so the choice is workflow, not security.

The implementation grants each environment's EC2 instance role `s3:GetObject` on exactly one environment-scoped S3 configuration bucket. The predeploy hook simply tries each candidate bucket in turn; IAM permits exactly one to succeed. The bucket that responds is the environment. As a defense in depth, the configuration file fetched from that bucket also contains an explicit environment marker, and the hook cross-checks it against the bucket name on each predeploy run. If they disagree, the instance refuses to start.

11.5 Per-environment configuration content

The per-environment configuration consists of a small number of files: a properties file for the application, a logging configuration, and two configuration files for the application server. They are uploaded once per environment from a secured machine and are never written by the deployment pipeline. The release artifact is bit-identical across all environments; the environment-specific data lives only in the configuration buckets. Database passwords, mail-transport credentials, and similar true secrets are part of the properties file and are protected by the configuration bucket's encryption-at-rest and access controls. Because the artifact carries no secrets and the configuration buckets are written out of band, the mechanism cleanly separates release authority from secret custody: triggering a deployment requires only the ability to upload an artifact, not access to any environment's secrets, so the role that releases and the role that holds production secrets need not be the same person.

The same configuration keys appear in every environment with different values, so the artifact stays environment-neutral while each environment supplies its own database connection pool settings, database and mail-transport credentials, service endpoints, and an environment marker. The mechanism here is a protected per-environment S3 bucket; a managed secrets service (AWS Secrets Manager, SSM Parameter Store) is an equally valid alternative.



```
   DEV          UAT          PROD     each instance reads its own
    |            |            |       configuration at startup
    v            v            v
config       config       config     per-environment config source:
source       source       source     protected S3 bucket, or AWS Secrets
                                       Manager / SSM Parameter Store

same keys, different values per environment (config.properties):
  db.url, db.pool.maxActive, db.pool.timeout . db connection pool
  db.user, db.password ..................... db credentials   (secret)
  smtp.host, smtp.user, smtp.password ....... mail transport   (secret)
  cdn.distributionId, *.bucket .............. CDN / storage identifiers
  environment = DEV | UAT | PROD ............ env marker (cross-checked)
```

12 Results

At the time of writing the deployed application comprises approximately 90,000 lines of source code; the measurements below therefore describe a substantial production codebase rather than a minimal example.

The first fully autonomous end-to-end deployment in the development environment took approximately 90 seconds from `upload completed` on the release master's machine to the running application reporting the new version label on its status endpoint. The breakdown was:

- S3 `ObjectCreated` event delivery and CodeBuild project startup: ≈ 5 seconds.
- CodeBuild project: artifact download, static-asset synchronization, CloudFront invalidation request, Elastic Beanstalk source-bundle assembly and upload, `CreateApplicationVersion`, `UpdateEnvironment`: ≈ 30 seconds.
- Elastic Beanstalk rollout of a single-instance environment: ≈ 28 seconds. Single instance only; multi-instance rollouts add per-instance time, but the per-instance work is unchanged.
- Post-rollout health check and status-endpoint verification: ≈ 5 seconds.

13 Discussion

The reported timing shows only that the autonomous pipeline is fast enough to remove deployment scheduling as a planning constraint; the absolute number is not itself the interesting quantity, since a single build is fast under either model. The two models diverge at fleet scale and on rollback, neither of which this single-instance trial measures. Under Model A the build and dependency-resolution cost is paid once *per instance* and grows with fleet size, and rolling back means rebuilding a previous revision on every instance with no guarantee the rebuild reproduces the prior bytes; under Model B the build is paid once and rollback is the re-selection of an artifact already present, in seconds and without recompilation. Quantifying that difference, the per-instance build cost multiplied across a fleet, and the rollback time of selection versus rebuild, is the substance of the comparison, and is left to the planned multi-instance and load evaluation.

The CodeBuild naming mismatch noted in Methods is a common point of confusion for engineers new to the pattern, and a detail worth flagging in any documentation derived from it.

14 Limitations

The case study covers a single production application on a single commercial cloud platform; multi-region rollouts, blue-green and canary deployment overlays, and stateful-workload migrations are out of scope. Database schema migrations are operated as a separate manual step in this implementation; integrating them into the autonomous pipeline introduces correctness risks that are not addressed here. The regulatory section names major regimes but does not attempt the exhaustive control-by-control mapping that a formal compliance package would require.

15 Conclusion

A deployment pipeline whose only manual step is the upload of a versioned artifact to an S3 bucket can be operated reliably under a small set of conditions: the environment identifier must be carried by IAM scoping rather than by mutable side channels, and the per-environment configuration must be pulled by the instance at start from a single, authoritative source. Under these conditions, end-to-end deployment is fast enough to remove deployment scheduling as a planning constraint, and this class of configuration drift across a fleet is structurally prevented, because every instance fetches its configuration from one environment-scoped source on every deploy rather than relying on files placed by hand.

Part I argued that artifact promotion is the right control model. Part II reports that the deployment mechanism is small and that its design generalizes cleanly to the production migration that follows.

Author Note

The first author's prior work spans regulated domains treated in this paper. In a Canadian provincial health system he produced and distributed hashable binary packages, written in C and Java, to hospitals, health-care facilities, and medical-education institutions, under the province's health-information-protection law and federal privacy legislation (the Canadian counterparts to the U.S. HIPAA Security Rule), which made a fixed, hashable binary a precondition for distribution. He also built financial rule-engines, among them a provincial consolidation of financial and statistical accounts that ranked first among provinces for data quality. This experience informs the paper's treatment of artifact identity and integrity as regulatory preconditions rather than engineering preferences.

The second author implemented the deployment infrastructure described in Part II: the event-triggered deployment runner, the managed platform-as-a-service environment, and the IAM scoping. He holds a PhD and is a certified cloud- and information-security practitioner (CISSP, CCSP); the supply-chain and control-domain arguments developed here are central to that practice.

References

[1] Gavrilov, V. (2026). *Artifact Promotion as a Control Model for Stable Cloud Deployment: Build on Target Machines vs. Build Once and Promote Artifacts.* Zenodo. `https://doi.org/10.5281/zenodo.20451077`

[2] Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation.* Addison-Wesley.

[3] Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps.* IT Revolution Press.

[4] Wiggins, A. (2017). *The Twelve-Factor App.* Heroku. https://12factor.net/

[5] National Institute of Standards and Technology. (2020). *Security and Privacy Controls for Information Systems and Organizations.* NIST Special Publication 800-53, Revision 5.

[6] Cybersecurity and Infrastructure Security Agency (CISA). (2025). *Supply Chain Compromise of Third-Party tj-actions/changed-files (CVE-2025-30066) and reviewdog/action-setup (CVE-2025-30154).* CISA Alert, 18 March 2025. GitHub Advisory GHSA-mrrh-fwg8-r2c3 (CVE-2025-30066), https://github.com/advisories/GHSA-mrrh-fwg8-r2c3.

[7] Ladisa, P., Plate, H., Martinez, M., & Barais, O. (2023). SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *2023 IEEE Symposium on Security and Privacy (SP).* https://doi.org/10.1109/SP46215.2023.10179304

[8] Ohm, M., Plate, H., Sykosch, A., & Meier, M. (2020). Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2020)*, LNCS 12223. Springer.

[9] European Union Agency for Cybersecurity (ENISA). (2021). *Threat Landscape for Supply Chain Attacks.*

[10] CVE Program and National Vulnerability Database. *Published CVE Record Counts by Year.* https://www.cve.org/about/Metrics

[11] Open Source Security Foundation (OpenSSF). (2023). *Supply-chain Levels for Software Artifacts (SLSA), Version 1.0.* https://slsa.dev/

[12] Torres-Arias, S., Afzali, H., Kuppusamy, T. K., Curtmola, R., & Cappos, J. (2019). in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes. In *28th USENIX Security Symposium.*

[13] Newman, Z., Meyers, J. S., & Torres-Arias, S. (2022). Sigstore: Software Signing for Everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS).* https://doi.org/10.1145/3548606.3560596

[14] Reproducible Builds Project. *Reproducible Builds: An Independently-Verifiable Path from Source to Binary Code.* https://reproducible-builds.org/